

Our best price of the year. **Get 20% off new memberships** for a limited time.

25 Understanding Chrome Extensions OAuth

7 min read · Sep 25, 2024



M2K Developments

Follow



Listen



Share



By using the `chrome.identity` API it enables use to identify/retrieve signed-in users, authenticate via Google OAuth or other social OAuth options, get access token tokens and more.

In this article we are going to cover how to initial a login sequence for Facebook and Google using the `chrome.identity` API. You see the official documentation on the API here:

<https://developer.chrome.com/docs/extensions/reference/api/identity>

How to declare it in Manifest.json

```
{
  ...
  "permissions" : [
    "identity"
  ],
  ...
}
```

How to get Account Details

If you want want the details for a user signed into chrome. First add the `identity.email` permission in the manifest along side the `identity` permission.

```
{
  ...
  "permissions": [
    "identity",
    "identity.email"
  ],
  ...
}
```

Next is by call the the `chrome.identity.getProfileUserInfo` function. If the user is signed it will return the `email` and `gaia id` . There are only 2 options for `accountStatus` `ANY` or `SYNC` .

```
chrome.identity.getProfileUserInfo({ accountStatus: "ANY" }, (user) => {
  console.log(user.email);
  console.log(user.id);
})

// or async/await version

const user = await chrome.identity.getProfileUserInfo({ accountStatus: "ANY" });
console.log(user.email);
console.log(user.id);
```

To Detect if an account was change you can use the event listener `onSignInChanged`

```
// Fired when signin state changes for an account on the user's profile.
chrome.identity.onSignInChanged.addListener((account, signedIn) => {
  if (signedIn) {
    console.log(account.id);
  }
})
```

How to Launch a Login Flow

We'll use Facebook as an example.

Prerequisites

You'd at least need to know the basics of how OAuth 2.0 works and have a general understanding of what `access_token` , `refresh_token` , `redirect_url` , `client_id` and `client_secrets` are. Or better yet have implemented a OAuth Login on website or webapp.

Start the Dialog

Ever social login requires the user to go the oauth page to start the login process. The `identity` API aids us by giving us the `chrome.identity.launchWebAuthFlow` function that does exactly just that. It will launch a web view window to the oauth page we want.

```
const client_id = '';
const redirectUrl = '';
const authUrl = ``;

// https://developer.chrome.com/docs/extensions/reference/api/identity#method-launchWebAuthFlow
```

```
const options = {  
  // The URL that initiates the auth flow.  
  url: authUrl,  
}  
  
// Start web view  
chrome.identity.launchWebAuthFlow(options, (responseUrl) => {  
  if (responseUrl) {  
    // Get tokens,code or othe values from url  
  }  
});
```

Okay let's explain what is happening here. For the a conventional oauth login flow the user will have to be direct to a URL that usually contains the `client_id`, `scopes` and `redirect_url` in any format like this

```
const client_id = '';  
const redirectUrl = '';  
const scopes = ''  
const authUrl = `https://app.example.com/oauth?client_id${client_id}&redirectUrl=${redirectUrl}&scopes=${scopes}`;
```

The `client_id` and `scopes` are values you can getting from the platform you are registering for. However a question you might have is *“what do I put in the redirect Url?”*

Redirect Url

As the name implies the redirect url is the url the user is redirected to when the login for is complete and usually code and tokens are attached along side the url when the user is redirected.

```
https://redirecturl.com?code=SomeRandomCode&token=0rSomeRandomToken
```

The redirect url is usually something you'd know before hand it is a requirement for you need to enter in the social platform that you want to make a login for.

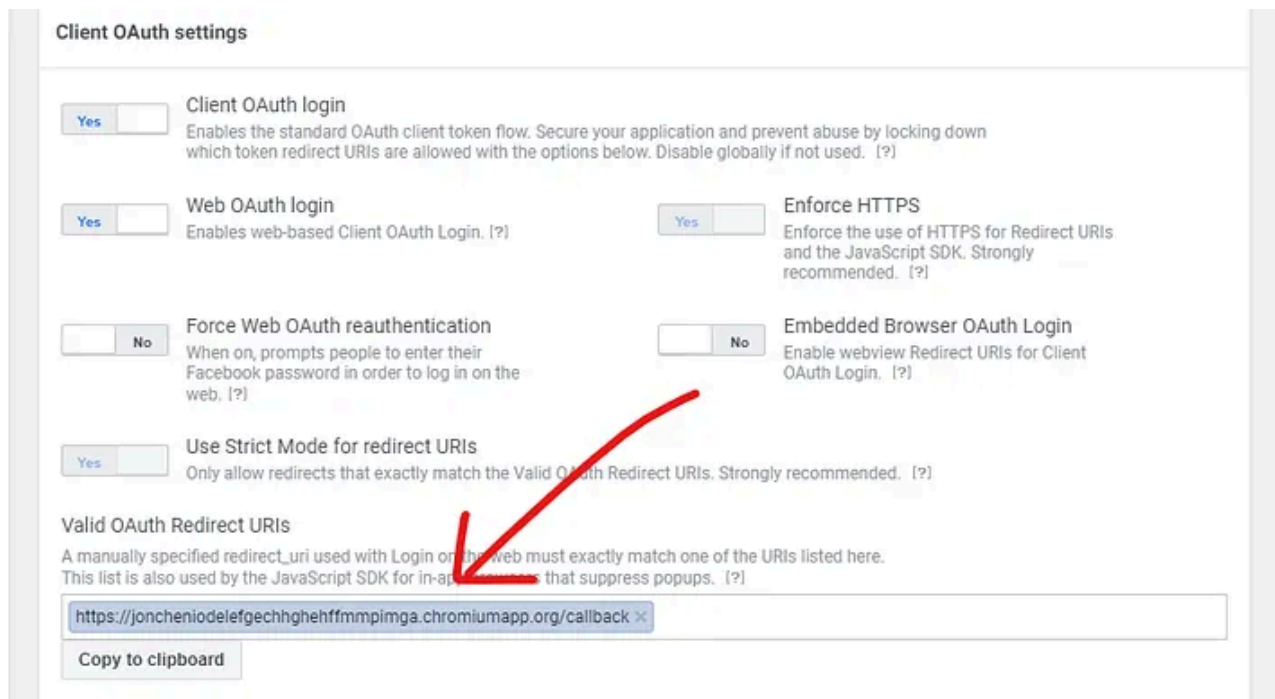
Get M2K Developments's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

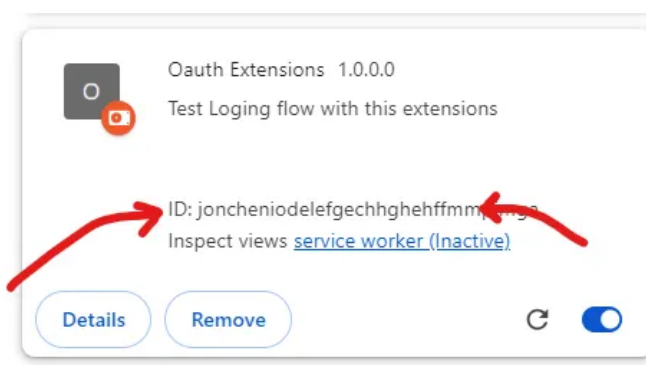
Subscribe

For chrome extensions the is a special format used `https://<app-id>.chromiumapp.org/*` . And this is the redirect url that you will need to but in the social platform you are targeting.



An example of adding the redirect url in the Facebook platform.

Where to get the app id



Showing the app id for a chrome extension.

```
const id = chrome.runtime.id
const redirectUrl = `https://${id}.chromiumapp.org/callback`

// or use the getRedirectURL
const redirectUrl = chrome.identity.getRedirectURL('callback');
```

What happens after the web view reaches the redirect url

When the web view reaches this redirect extension will close automatically and the callback function is called. So let us review that code again but will use Facebook as an example

```
const clientId = '12345*****';
const redirectUrl = chrome.identity.getRedirectURL('callback');
const authUrl = `https://www.facebook.com/v20.0/dialog/oauth?` +
  `client_id=${clientId}` +
  `&redirect_uri=${redirectUrl}` +
  `&state=randomState`

const options = {
  url: authUrl, // The URL that initiates the auth flow.
}
```

```
// Start web view
chrome.identity.launchWebAuthFlow(options, (responseUrl) => {
  if (responseUrl) {
    // Get tokens,code or othe values from url
  }
});
```

The `responseUrl` will usually contain the codes and token that you need to make authenticated requests. Although you'd need to use some JavaScript trickery to get them as there are appended as query params

<https://904823.chrome..chromiumapp.org/callback?token=whatyouwant>

Google OAuth

You can able the sample methodology for the Google OAuth flow, however the are special functions in the `chrome.identity` for Google OAuth which make it more convient. It done require some setting up to do though.

[Here](#) is a guide a how to set up your Google Console Project for chrome extensions.

<https://developer.chrome.com/docs/extensions/how-to/integrate/oauth>. So go do that first.

You're back? Alright for 10% of y'all who did it. You can now use the `chrome.identity.getAuthToken` function initiate a special popup for Google OAuth. and the callback has the access token from authenticate requests.

```
// For more options - https://developer.chrome.com/docs/extensions/reference/api/identity#type-TokenDetails
const options = {
  // show popup if necessary
  interactive: true,

  // by default it uses the ones declared in manifest.json
  scopes: []
};
chrome.identity.getAuthToken(options, (accessToken) => {

  // make request with access token

});
```

To De-authorizes the user

```
chrome.identity.clearAllCachedAuthTokens(() => {

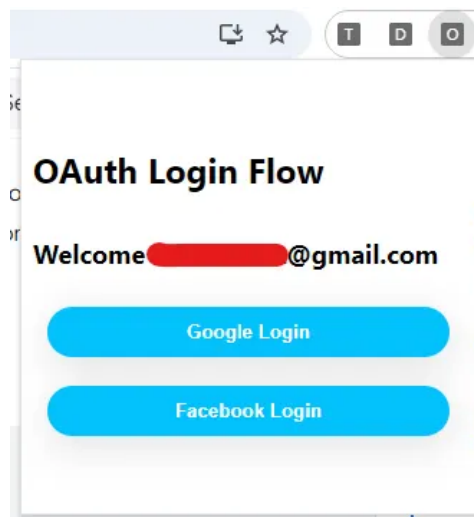
});

// or async/await version
await chrome.identity.clearAllCachedAuthTokens()
```

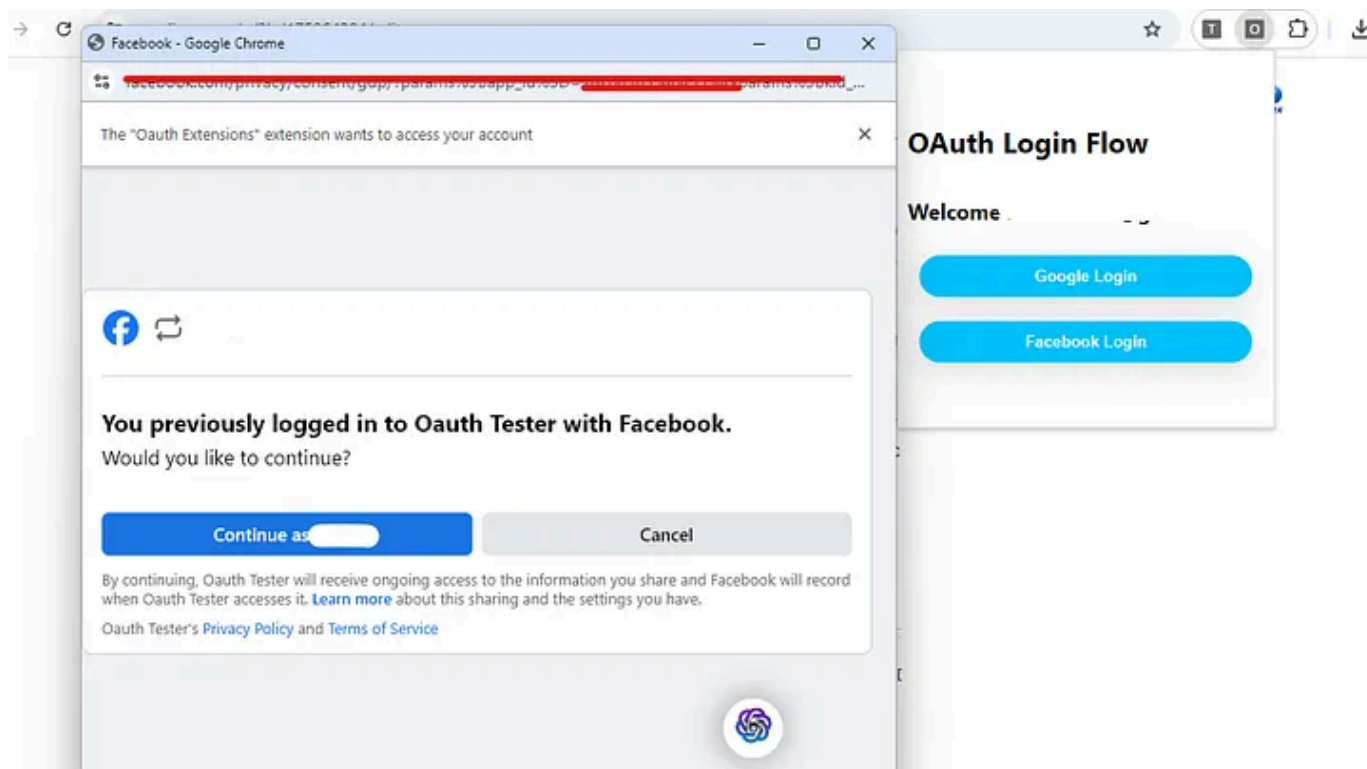
Sample Project

In this project we are going to build a base framework for an extension with a Google and Facebook login page and a main page once they have logged in.

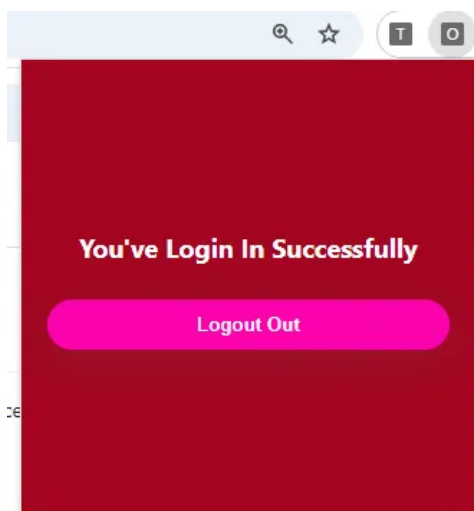
You can access the source code to the project [here](https://github.com/BuildChromeExtensions/chromeGoogleOAuth): <https://github.com/BuildChromeExtensions/chromeGoogleOAuth>



index.html — Login Page



Facebook Oauth Screen



main.html main page

We will need 5 file for this one: `manifest.json`, `index.html`, `scripts.js`, `main.html` and `main.js`.

You'll need to replace the `client_id` for **Google** in `manifest.json` and the `client_id` for **Facebook** in the `script.js` with your own.

manifest.json

```
{
  "name": "OAuth Extensions",
  "description": "Test Logging flow with this extensions",
  "version": "1.0.0.0",
  "manifest_version": 3,
  "oauth2": {
    "client_id": "*****.apps.googleusercontent.com",
    "scopes": [
      "profile",
      "email"
    ]
  },
  "action": {
    "default_popup": "index.html"
  },
  "permissions": [
    "identity",
    "identity.email"
  ]
}
```

index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    body {
      width: 300px;
      height: 300px;
      display: flex;
      flex-direction: column;
      justify-content: center;
    }

    button {
      background: #03c2fc;
      color: #ffffff;
      padding: 10px 20px;
      border: none;
      cursor: pointer;
      box-shadow: rgba(100, 100, 111, 0.2) 0px 7px 29px 0px;
      transition: all 0.4s;
      border-radius: 30px;
      font-weight: 700;
      margin: 10px;
    }

    button:hover {
      background: #026482;
    }
  </style>
</head>

<body>
  <h1>OAuth Login Flow</h1>
```

```

<h2></h2>
<button id="btn-google">Google Login</button>
<button id="btn-facebook">Facebook Login</button>
<script type="text/javascript" src="./scripts.js"></script>
</body>

</html>

```

scripts.js

```

chrome.identity.getProfileUserInfo({ accountStatus: "ANY" }, (user) => {
  console.log(user);
  if (user.email) {
    document.querySelector('h2').textContent = `Welcome ${user.email}`;
  }
})

document.getElementById('btn-google').onclick = async () => {

  // For more options - https://developer.chrome.com/docs/extensions/reference/api/identity#type-TokenDetails
  const options = {
    interactive: true, // show popup if necessary

    // by default it uses the ones declared in manifest.json
    // scopes: []

    /*The account ID whose token should be returned.
    If not specified, the function will use an account
    from the Chrome profile: the Sync account if there
    is one, or otherwise the first Google web account.
    */

    // account: {
    //   id: ""
    // }
  }
  chrome.identity.getAuthToken(options, (accessToken) => {
    console.log(accessToken);

    chrome.action.setPopup({ popup: "main.html" });
    window.location.href = "main.html";

  });
}

document.getElementById('btn-facebook').onclick = () => {

  const clientId = '*****';
  const redirectUrl = chrome.identity.getRedirectURL('callback');
  const authUrl = `https://www.facebook.com/v20.0/dialog/oauth?` +
    `client_id=${clientId}` +
    `&redirect_uri=${redirectUrl}` +
    `&state=randomState`

  // https://developer.chrome.com/docs/extensions/reference/api/identity#method-launchWebAuthFlow
  const options = {
    // The URL that initiates the auth flow.
    url: authUrl,
    interactive: true,
    abortOnLoadForNonInteractive: true,
    timeoutMsForNonInteractive: 60000,
  }

  // Start web view
  chrome.identity.launchWebAuthFlow(options, (responseUrl) => {
    console.log(responseUrl)
    if (responseUrl) {
      // Get tokens, code or other values from url
      const containsCode = responseUrl.indexOf("code") !== -1;

```



```
    if (containsCode) {  
      chrome.action.setPopup({ popup: "main.html" });  
      window.location.href = "main.html";  
    }  
  }  
});  
}
```

main.html

```
<!DOCTYPE html>  
<html lang="en">  
  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Document</title>  
  <style>  
    body {  
      width: 300px;  
      height: 300px;  
      display: flex;  
      flex-direction: column;  
      text-align: center;  
      justify-content: center;  
      background: #a30520;  
      color: white  
    }  
  
    button {  
      background: #fc03ad;  
      color: #ffffff;  
      padding: 10px 20px;  
      border: none;  
      cursor: pointer;  
      box-shadow: rgba(100, 100, 111, 0.2) 0px 7px 29px 0px;  
      transition: all 0.4s;  
      border-radius: 30px;  
      font-weight: 700;  
      margin: 10px;  
    }  
  </style>  
</head>  
  
<body>  
  <h2><b>You've Login In Successfully</b></h2>  
  <button>Logout Out</button>  
  <script type="text/javascript" src="./main.js"></script>  
</body>  
  
</html>
```

main.js

```
document.querySelector('button').onclick = () => {  
  // Logged  
  chrome.identity.clearAllCachedAuthTokens(() => {  
    chrome.action.setPopup({ popup: "index.html" });  
    window.location.href = "index.html";  
  });  
}
```

Chrome Extension

OAuth

Identity

Login

Authentication



Follow



Written by M2K Developments

29 followers · 9 following

Responses (1)



Write a response

What are your thoughts?



Marian C.

Jan 7



Nicely explained and to the point



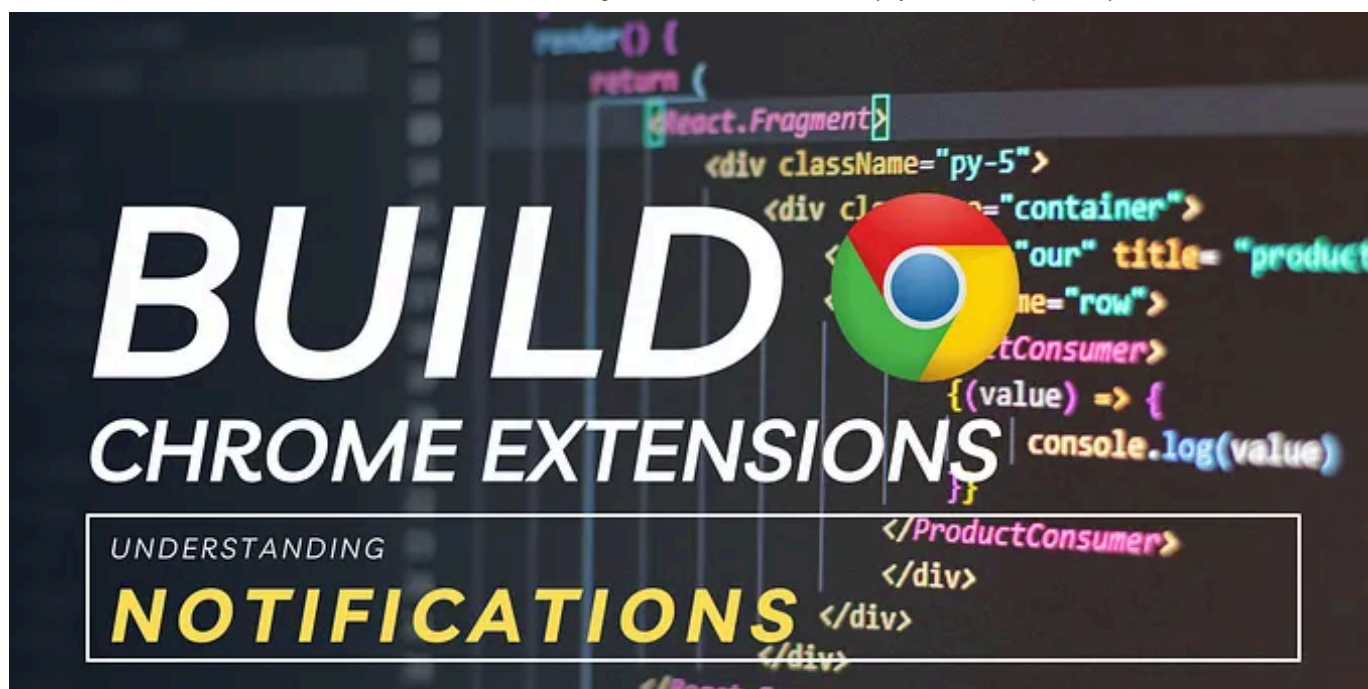
1



1 reply

[Reply](#)

More from M2K Developments



 M2K Developments

14 Understanding Chrome Extensions Notifications

The chrome notifications API is responsible for displaying different types of what is called rich notifications in your users' system tray.

Sep 25, 2024  2



 M2K Developments

4 Understanding Chrome Extensions Background Scripts

Simply put background scripts are service workers JavaScript files that handle then extensions main events. They operate independent of a...

Sep 4, 2024  4





M2K Developments

23 Understanding Chrome Extensions Scripting

The chrome.scripting API is used to programmatically inject CSS and JavaScript to alter the behavior and appearance of the websites. Think...

Sep 24, 2024 3



Open in app

Sign up

Sign in

Medium

Search



M2K Developments

11 Understanding Chrome Extensions Context Menus

Context Menu is the menu that appears when a user right clicks (alternative clicks) on an element in a webpage.

Sep 25, 2024 3



See all from M2K Developments

Recommended from Medium

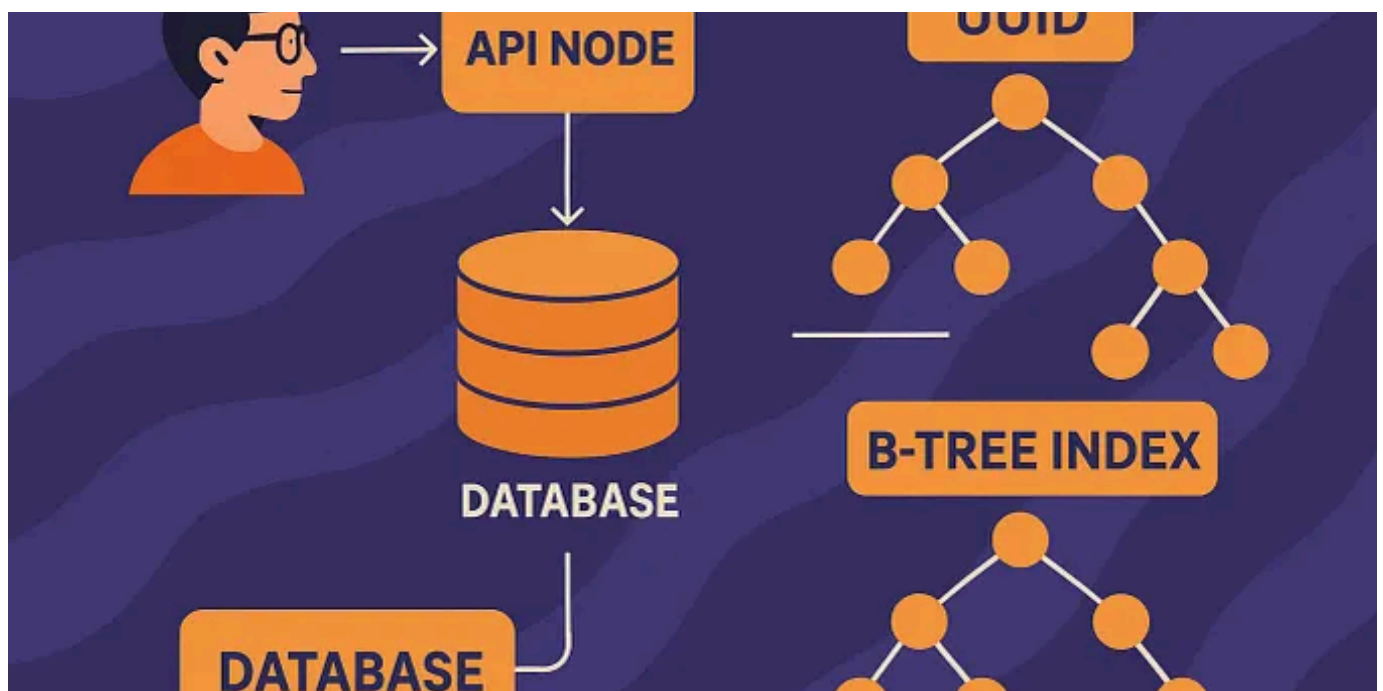


In Mr. Plan B Publication by Monika sharma

Hidden in Headers: The Power of Misused Referer, Origin, and Host Headers

How Three Common HTTP Headers Can Break Authentication, Hijack Sessions, and Lead to Full Account Takeover

Jun 11 2

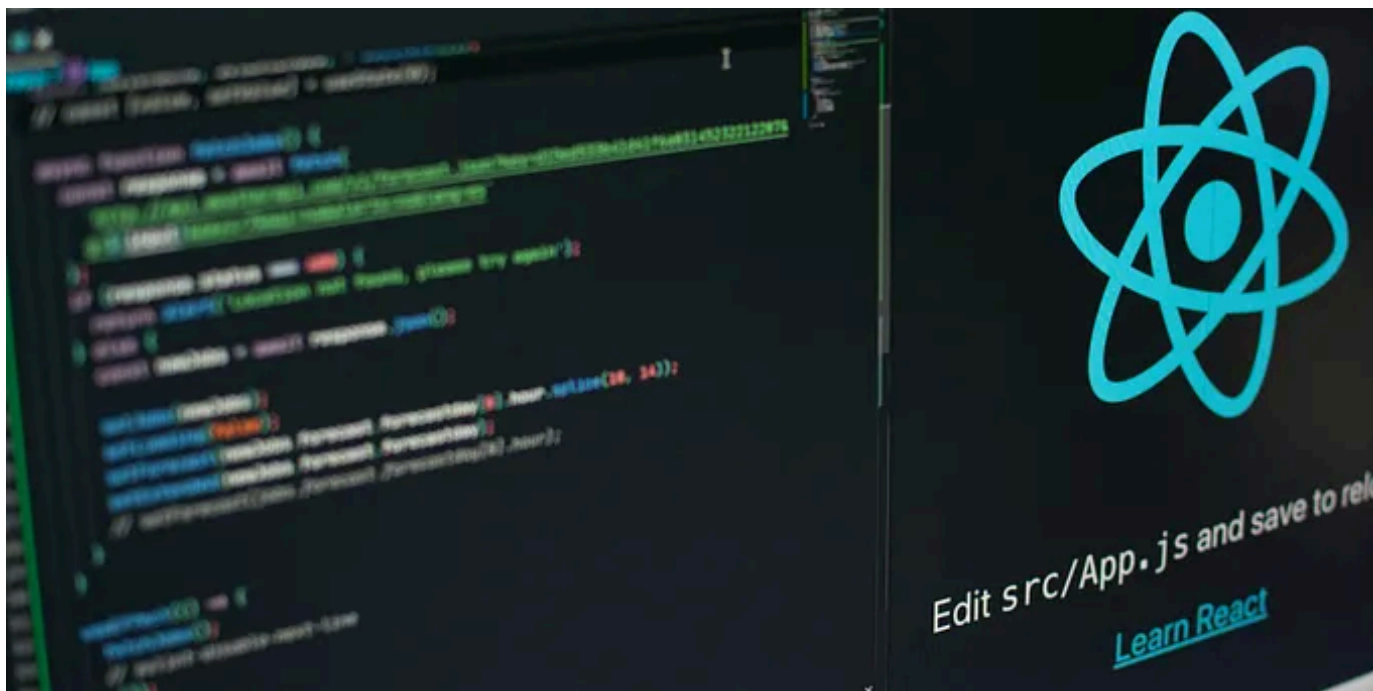


The Concurrent Mind

Stop Using UUIDs as Primary Keys. It Is Killing Your Database Throughput

Your database is not slow because it is old. It is slow because every new row lands in a random place that your CPU cannot predict.

Nov 24 265 11

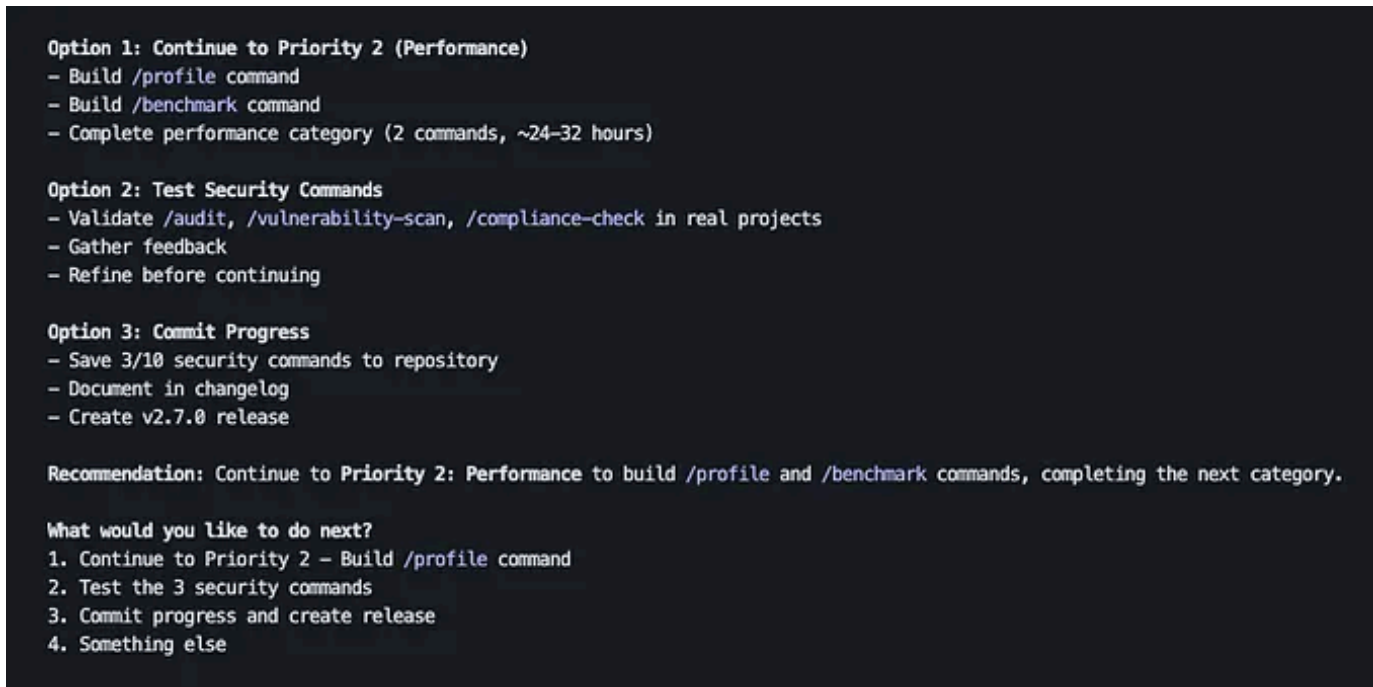


Kennedy Owusu

REST API Architecture in React: A Production Guide

How to structure your API layer for maintainability, testability, and scale

Jun 23 6 2



Reza Rezvani

10 Claude Code Commands That Cut My Dev Time 60%: A Practical Guide

Custom slash commands, subagents, and automation workflows that transformed my team's productivity—with copy-paste templates you can use

Nov 20 465 16



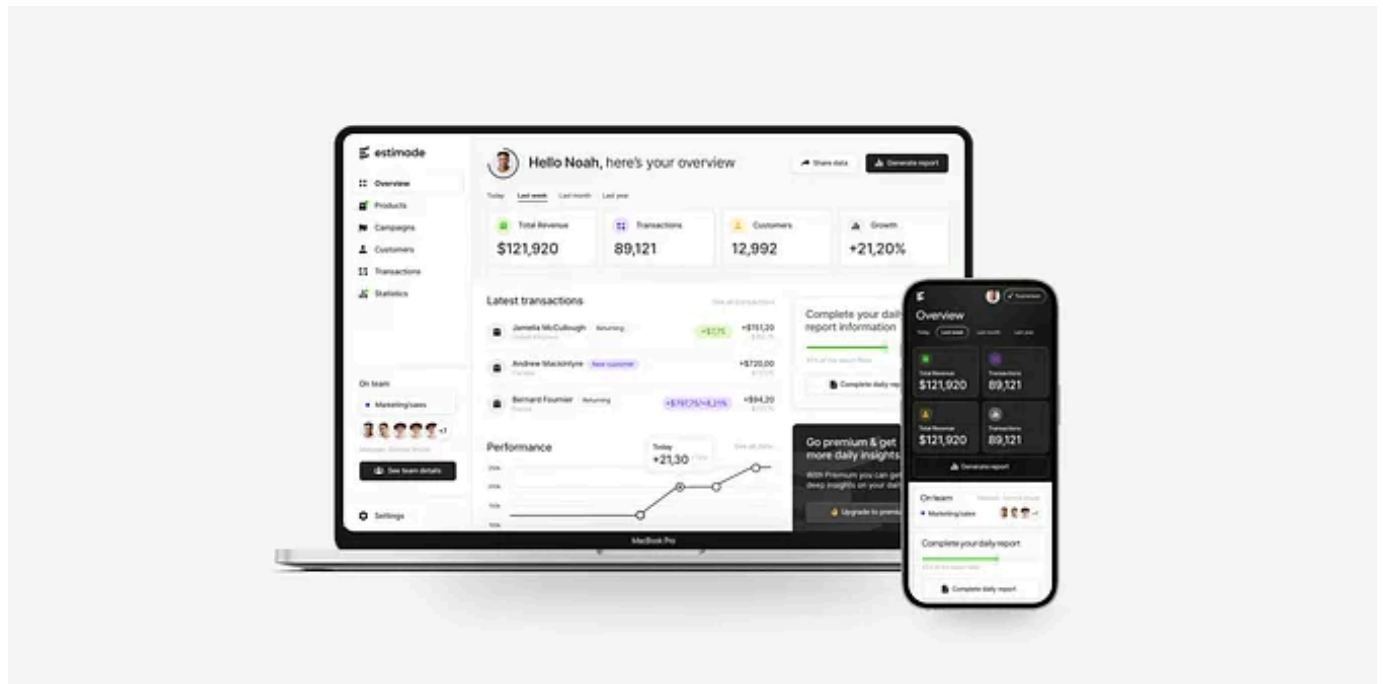


 In Level Up Coding by Anmol Baranwal

11 Powerful APIs for Your Next Project

Nobody wants to build things from scratch. That's why APIs exist.

★ Oct 24 🖱️ 711 💬 14



 Michal Malewicz

The End of Dashboards and Design Systems

Design is becoming quietly human again.

★ 6d ago 🖱️ 849 💬 28



See more recommendations